AUTONOMOUS INCIDENT TRIAGE & ROOT CAUSE PLATFORM

JiraPilot AI

Turn Jira Issues into Actionable Investigations

Your Autonomous Investigation Copilot

JiraPilot AI helps software teams organize incoming issues, inspect log evidence, discover relevant repositories, and prepare clearer investigation updates automatically.

Built by Runtime Terrors • Hackathon 2026 • Live Integration with Jira Cloud & EmailJS

THE INVESTIGATION BOTTLENECK

When an issue arrives, where do developers start?

1. Missing Context
[UNPREPARED TICKETS]

- Tickets arrive without critical reproduction details.
- Missing API endpoints, HTTP verbs, and environments.
- Developers must repeatedly ping reporters for missing logs.
- Back-and-forth comments cause hours of initial delay.

2. Scattered Logs
[UNORGANIZED EVIDENCE]

- Diagnostic evidence is fragmented across multiple server logs.
- Files contain tens of thousands of lines of noise.
- No standardized correlation between stack trace and ticket.
- Manual keyword grep searches waste engineering hours.

3. Complex Topologies
[OPAQUE ARCHITECTURE]

- Single client errors mask deep multi-microservice cascades.
- Developers manually trace HTTP 500/503 errors across hops.
- Difficult to distinguish root failure from secondary symptoms.
- Lack of clear, visual request journey maps.

4. Communication Drag
[MANUAL HANDOFFS]

- Locating the right Git codebase requires tribal knowledge.
- Writing diagnostic summaries for Jira is inconsistent.
- On-call alerts lack immediate root cause hypotheses.
- Junior engineers struggle to find where to begin debugging.

INDUSTRY REALITY

The challenge is not just fixing the bug

THE REPEATED MANUAL INVESTIGATION JOURNEY TODAY

Read Ticket → Ask for Details → Find Logs → Trace Error Across Services → Find Code → Share Findings

Frequent Context Switching

- Engineers are pulled away from active feature sprints to triage unvalidated tickets.
- Mental reset time after interrupted debugging impacts productivity.
- Repetitive triage creates operational fatigue.

Inconsistent Diagnostic Handoffs

- Different developers write different ticket comments.
- Critical evidence like correlation IDs or log line references are lost.
- Handoffs between time zones and shifts become high-friction.

Organize, Don't Replace

[HUMAN-IN-THE-LOOP]

- JiraPilot AI does not attempt to write speculative production code.
- It organizes evidence so engineers start debugging with full facts.
- Human engineering judgment remains in complete control.

THE PLATFORM SOLUTION

Meet JiraPilot AI: Your Autonomous Investigation Copilot

1. Ticket Readiness Engine
[AUTOMATED TRIAGE]

- Deterministic 100-point scoring matrix.
- Detects missing endpoints, environments, logs, and payloads.
- Prepares polite clarification comments automatically.

2. Log Telemetry Parser
[EVIDENCE EXTRACTION]

- Deterministic regex extraction of HTTP status, methods, endpoints.
- Correlation ID and line-referenced evidence gathering.
- Automatic credential and secret redaction before processing.

3. Holographic API Flow Map
[TOPOLOGY MAP]

- Interactive visual request path from Client to Response.
- Instant visual failure node isolation (e.g. HTTP 503).
- Synchronized request timeline and line-referenced log viewer.

4. Repository Discovery
[CODE NAVIGATION]

- Matches affected services against configured Git codebases.
- Provides approved, copyable git clone commands.
- Eliminates tribal knowledge search for repositories.

5. 7-Section Investigation Report
[STRUCTURED RCA]

- Executive summary, failure points, and probable root causes.
- Confidence ratings, evidence base, and developer action steps.
- Reproduction cURL commands and test case blueprints.

6. Live Multi-Channel Handoff
[JIRA & EMAILJS]

- One-click automated structured comment posting to Jira Cloud.
- Animated responsive HTML triage email dispatch via EmailJS.
- Live webhook integration and centralized audit logging.

END-TO-END WORKFLOW

From a Jira Issue to a Clearer Investigation

PATH A: Incomplete Ticket (< 70 Readiness)

[CLARIFICATION WORKFLOW]

- Ticket ingested via Jira Cloud webhook or sync.
- Readiness Engine scores ticket across 6 criteria (e.g., Score: 10/100).
- Status set to WAITING_FOR_INFORMATION.
- System prepares polite clarification comment listing exactly what is needed (Endpoint, Logs, Payload).
- Audit trail logged; developer is spared from manual context gathering.
- When reporter provides logs or payload, system auto-resumes.

PATH B: Ready Ticket (≥ 70 Readiness)

[FULL ROOT CAUSE WORKFLOW]

- Ingested ticket verified READY (Score: 100/100).
- Log Analysis Engine parses raw telemetry & decompresses attachments.
- Extracts API endpoints, correlation IDs, timestamps, and stack traces.
- Constructs Interactive API Flow Map isolating the exact failure node.
- Matches relevant Git repository & approved clone command.
- Synthesizes 7-section structured RCA report.
- Posts structured investigation comment to Jira Cloud & dispatches EmailJS alert.

DIAGNOSTIC READINESS EVALUATION

Feature 1: Start With the Right Information

Example: Ticket NOVA-1001

[UNPREPARED INPUT]

- Ticket Title: 'User provisioning fails intermittently'
- Description: 'Users cannot complete signup flow. Please fix.'
- Missing Fields Detected:
- ✗ API Endpoint & HTTP verb
- ✗ Application Server Logs
- ✗ Sample Request Payload (JSON)
- ✗ Environment (Prod/Stage)
- Initial Readiness Score: 10 / 100
- Status: WAITING_FOR_INFORMATION

Readiness Checklist Matrix

[EVALUATION ENGINE]

- Description: Verified (Summary present)
- API Details: Checked against OpenAPI schemas
- Server Logs: Validated format (.log, .txt, .gz)
- Request Payload: Checked for valid JSON
- Environment: Checked against known tiers
- Error Pattern: Checked for exception keywords
-
- Scoring weights are customizable via JiraPilot Policy Service.

Automated Clarification Comment

[AUTONOMOUS TRIAGE]

- Generated for Jira Cloud:
- 'Hello! Our triage engine found a readiness score of 10/100. To initiate deep root cause investigation, please provide:
- • API Endpoint & Method
- • Server Logs
- • Sample Payload (JSON)'
-
- Once updated, JiraPilot automatically resumes triage.

LOG TELEMETRY CORRELATION

Feature 2: Turn Raw Logs Into Useful Evidence

THE FLIGHT RECORDER ANALOGY

"Logs are like the flight recorder of software. JiraPilot AI extracts the critical telemetry without the noise."

Raw Telemetry Input

[INPUT LOG]

- 10,000+ lines of application logs.
- Contains debug lines, heartbeats, and stack traces.
- Decompresses .gz and .log.gz files automatically.
- Enforces safety policy limit (up to 50,000 lines).

Extracted Telemetry Node

[ISOLATED FACTS]

- Endpoint: POST /api/v1/resources/allocate
- Status: HTTP 503 Service Unavailable
- Correlation ID: CORR-DEMO-1001
- Timestamp: 2026-10-10 10:24:32
- Service: resource-inventory-service
- Stack Frame: JpaResourceRepository.save():94

Evidence vs Hypothesis

[EXACT EVIDENCE]

- Observed Fact: HTTP 503 and SocketTimeoutException recorded on line 82.
- Possible Cause Hypothesis: Downstream socket read timeout after 5000ms boundary breach.
- Confidence Score: 88% (HIGH).
- Always labeled as a hypothesis for developer review.

HOLOGRAPHIC TOPOLOGY VISUALIZATION

Feature 3: See Where the Request Journey Breaks Down

Visual Request Topology

[INTERACTIVE MAP]

- Interactive holographic canvas inspired by futuristic observability consoles.
- Client Node: Initial ingress point with timestamp.
- Trunk Path: API 1 (/auth/login) → API 2 (/user/profile).
- Top Branch: Healthy service mesh (API 3 → API 5 → API 7).
- Bottom Branch: Isolated failure point (API 4 → API 6 fallback).
- Response Node: Final egress status.

Color-Coded Status Language

[INTUITIVE STATUS]

- Cyan / Blue Nodes: Verified successful APIs (HTTP 200) supported by telemetry.
- Red Glowing Hexagonal Node: Confirmed error evidence (HTTP 503) with pulsing aura.
- Amber Indicators: Inferred dependencies or degraded timeout warnings.
- Dashed Lines: Explicitly distinguished inferred architectural relationships.

Synchronized Investigation

[DEEP DIVE CONSOLE]

- Click any node to open the Right Details Panel (Overview, Line Logs, Payload, Related).
- Synchronized Request Timeline: 8-step sequence synchronized with node selection.
- Zoom, Pan, Fit-to-View, and Holo FX toggles.
- 1-Click Accessible List View for keyboard & screen-reader navigation.

CODE DISCOVERY & REPORTING

Feature 4: Move From Evidence to the Next Debugging Step

Repository Matcher

[GIT CODEBASE]

- Matches extracted services against configured Git codebases.
- Example: resource-inventory-service → https://github.com/northstar/resource-inventory-service.git
- Generates approved, copyable git clone commands.
- Eliminates guesswork about where the code lives.
- Clearly notes: Matching a repository is a starting point, not proof of root cause.

7-Section Jira Comment

[JIRA CLOUD]

- Posts structured comment to Jira Cloud:
- 1. Executive Summary
- 2. Failure Point & Root Cause
- 3. Diagnostic Evidence Base
- 4. Recommended Remediation & Fix
- 5. Developer Action Items
- 6. Reproduction cURL & Postman
- 7. Approved Clone Command

EmailJS Live Triage Alerts

[EMAILJS DISPATCH]

- Dispatches responsive, dark-mode animated HTML triage emails.
- Live pulsing radar beacon and telemetry breakdown matrix.
- macOS-style terminal reproduction box.
- Deep-link directly to JiraPilot incident workspace.
- Verified live with EmailJS API (Service: service_1ooa8fh).

SYSTEM ARCHITECTURE

How We Built It: Scalable Micro-Architecture

React Frontend
[PORT 3000]

- Framework: React 18 + TypeScript + Vite
- Styling: Tailwind CSS design tokens
- Visualization: Recharts + SVG Holographic Flow Canvas
- State: React state + zero external bloat
- Theme: Dark/Light mode support
- Port: 3000 (Hot Reload)

Spring Boot Backend
[PORT 8080]

- Framework: Java 21 + Spring Boot 3.2.5
- Database: H2 (Dev) / PostgreSQL (Prod) via JPA
- Responsibilities: Workflow coordination, Jira Cloud client, EmailJS client, audit logging
- Port: 8080 (Tomcat REST API)

FastAPI AI Service
[PORT 8000]

- Framework: Python 3.12 + FastAPI + Uvicorn
- Log Engine: Regex telemetry parser & evidence correlation
- Repository Matcher: Service-to-Git mapping
- AI Provider: Optional NVIDIA NIM LLM adapter with fallback
- Port: 8000

External Integrations
[INTEGRATIONS]

- Atlassian Jira Cloud: REST API v3 (Comments, Webhooks, Sync)
- EmailJS API: v1.0 REST API with browser-compatible headers
- NVIDIA NIM: Optional Kimi-k3 / Nemotron LLM endpoint
- Zero-Trust: Redacts secrets before any external dispatch.

BUSINESS IMPACT

Why JiraPilot AI Matters: Value, Differentiation, & Honesty

Impact by Stakeholder

[MEASURABLE VALUE]

- For Developers:
- • Less time spent checking tickets and grepping logs.
- • A clear starting point and copyable reproduction commands.
-
- For Teams:
- • Consistent 7-section investigation summaries.
- • Faster onboarding and smoother shift handoffs.
-
- For Organizations:
- • Standardized investigation workflows.
- • Reduced MTTR (Mean Time to Resolution).

What Makes It Different

[KEY DIFFERENTIATOR]

- Unified Pipeline: Connects ticket readiness, log parsing, topology visualization, repo discovery, and handoff in one tool.
- Deterministic First: Core extraction relies on verified logs, not hallucinated AI guesses.
- Human-in-the-Loop: Empowers engineers; does not attempt risky autonomous code edits.
- Non-Invasive: Integrates with existing Jira Cloud and Git repositories.

Honest Limitations

[HONEST LIMITATIONS]

- Incomplete Logs: Partial logs produce partial investigations.
- AI Hypotheses: Suggested root causes must be verified by engineers.
- Inferred Edges: If trace headers are missing, relationships are marked inferred.
- No Auto-Fixing: Does not execute code or modify production configurations.
- Requires Setup: Jira tokens and repo catalogs must be pre-configured.

THE BOTTOM LINE

Less Repetitive Triage. More Time to Solve.

JiraPilot AI — Your Investigation Copilot

1. Organize the Issue • 2. Understand the Evidence • 3. Decide the Next Step with Confidence

"Our goal is not to replace software engineers.
It is to help them spend less time organizing an investigation and more time solving the problem."

Thank You! • Questions & Live Demonstration